

A hand-drawn decorative border in black ink, featuring wavy lines and small loops at the corners and midpoints of the sides, framing the central text.

Git e GitHub

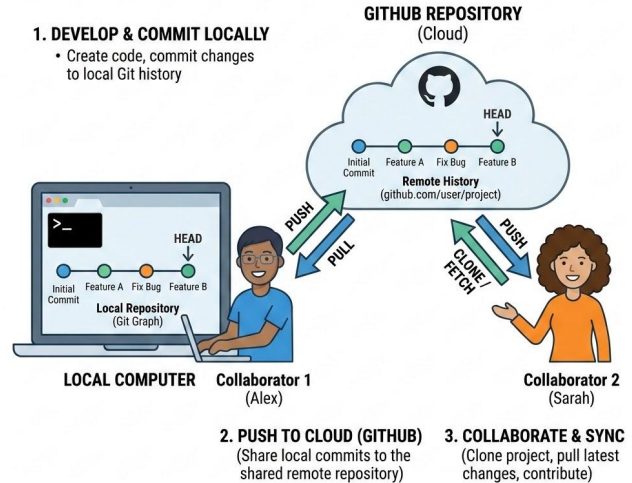
Cos'è il controllo di versione?

- Un sistema che registra le modifiche a file o insiemi di file nel tempo
- Permette di richiamare versioni specifiche in seguito
- Facilita la collaborazione tra sviluppatori
- Risolve i problemi di conflitti durante il lavoro parallelo
- Storia: dall'approccio manuale ai sistemi moderni

Git, GitHub e collaborazione

- In questa lezione vedremo come gestire un progetto in modo ordinato, tracciabile e collaborativo.
- Impareremo a usare Git per salvare la storia di un lavoro, tornare indietro quando serve, creare rami separati per nuove modifiche e integrare il lavoro di più persone.
- Vedremo poi GitHub come piattaforma per ospitare repository remoti, collaborare con issue e pull request, pubblicare documentazione minima e, come estensione pratica, pubblicare una semplice pagina web con GitHub Pages.
- Obiettivo finale della giornata: avere una repository GitHub funzionante, con README iniziale, cronologia sensata dei commit e una prova di collaborazione reale.

GIT & GITHUB WORKFLOW: COLLABORATION



Obiettivi concreti della giornata

Alla fine della lezione dovresti essere in grado di:

- spiegare perché il versioning è indispensabile in un progetto serio;
- distinguere working directory, staging area e repository;
- creare una repository Git locale;
- usare git init, git add, git commit, git status, git log;
- creare e usare branch;
- fare un merge semplice e gestire un conflitto minimo;
- collegare una repo locale a una repo remota su GitHub;
- usare clone, push, pull;
- creare issue e pull request;
- scrivere un README iniziale sensato;
- capire la differenza pratica tra licenza proprietaria, MIT, GPL e Apache 2.0;
- pubblicare una pagina statica semplice con GitHub Pages.

Perché serve il versioning

AKA: Perché non basta "salvare tante copie del file"

Senza versioning, un progetto degenera velocemente in cartelle con nomi come:

- `progetto_finale.docx`
- `progetto_finale_vero.docx`
- `progetto_finale_vero2.docx`
- `progetto_finale_definitivo_questa_volta.docx`

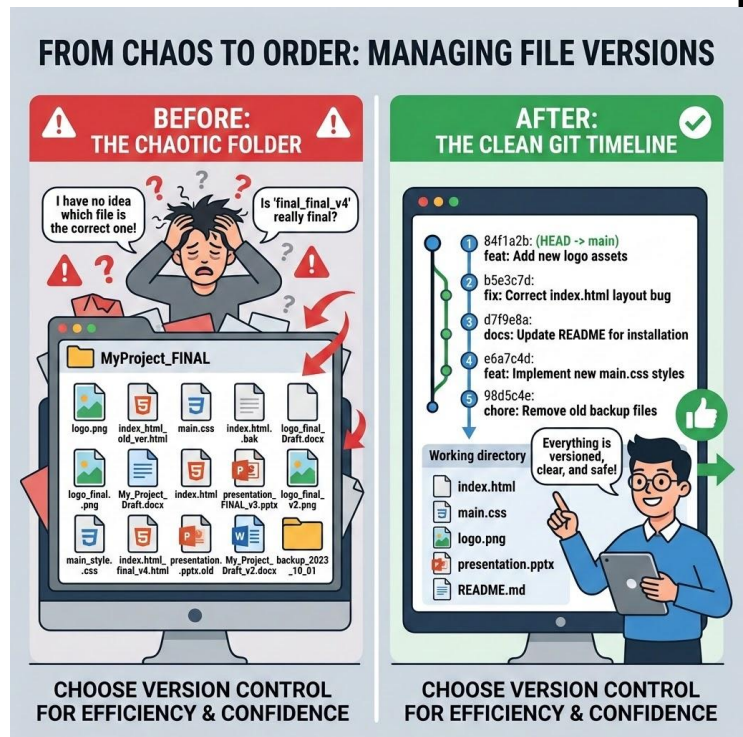
Questo approccio crea tre problemi seri:

1. **non sai con precisione cosa è cambiato;**
2. non sai chi ha cambiato cosa;
3. non sai tornare in modo pulito a uno stato precedente.

Il versioning risolve questi problemi perché salva una storia strutturata del lavoro.

Ogni **commit** rappresenta uno stato coerente del progetto.

In un team, questo permette di collaborare senza sovrasciversi a vicenda, discutere modifiche, rivedere il codice e recuperare errori.



Perché serve il versioning

AKA: Perché non basta "salvare tante copie del file"

Esempio concreto:

Due persone lavorano sullo stesso file HTML:

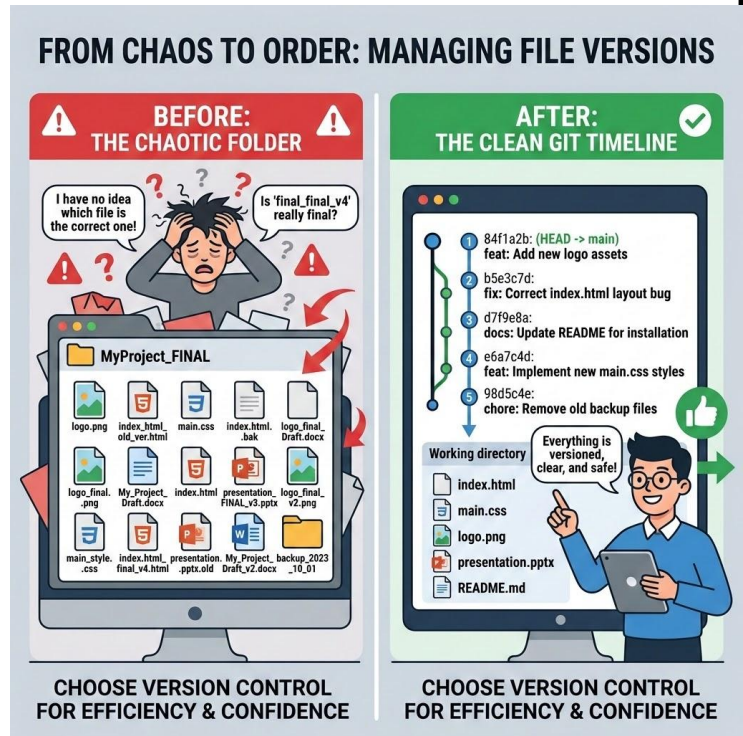
- una cambia il titolo e il layout;
- l'altra corregge testi e link.

Senza versioning è facile perdere pezzi di lavoro.

Con Git, ogni modifica può essere tracciata, confrontata e integrata.

Inoltre:

- backup e versioning non sono la stessa cosa. Il backup salva copie. Git salva storia, differenze, responsabilità, integrazione.
- Questo si può applicare anche a: documentazione, tesi, appunti tecnici, manuali, etc...



Sistemi centralizzati vs distribuiti

Centralizzato (SVN, CVS):

- Un server centrale contiene tutte le versioni
- Gli sviluppatori estraggono solo la versione corrente
- Richiede connessione al server per la maggior parte delle operazioni

Distribuito (Git, Mercurial):

- Ogni sviluppatore ha una copia completa dell'intero repository
- Possibilità di lavorare offline
- Maggiore ridondanza e sicurezza

Installazione e configurazione di Git

Installazione:

- Windows: <https://git-scm.com/download/win>
- macOS: `brew install git` o <https://git-scm.com/download/mac>
- Linux: `sudo apt-get install git` (Ubuntu/Debian)

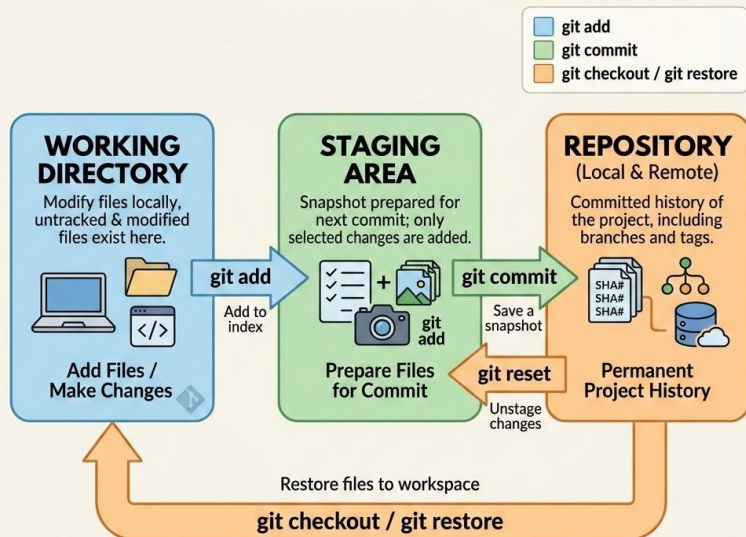
Configurazione iniziale:

```
1 git config --global user.name "Il tuo nome"
2 git config --global user.email "tua.email@esempio.com"
3 git config --global init.defaultBranch main
4
5
```


Il modello mentale di Git

Le tre aree da capire davvero

GIT DATA TRANSPORT COMMANDS & STAGES



Per usare Git bene serve capire tre aree distinte:

1. Working directory

È la cartella reale del progetto sul tuo computer. Qui modifichi file, crei cartelle, cancelli contenuti.

2. Staging area (index)

È l'area in cui prepari esattamente cosa entrerà nel prossimo commit. Non è ancora storia definitiva, è una selezione.

3. Repository

È la storia Git vera e propria, memorizzata localmente nella directory `.git`. Qui finiscono i commit.

Schema mentale corretto:

modifico file → seleziono con `git add` → salvo uno stato con `git commit`

Errore tipico da evitare: pensare che `git commit` prenda automaticamente tutto quello che hai modificato.

In realtà Git fa commit di ciò che è stato messo in staging.

Primo contatto: creare una repo locale

`git init` e il primo repository

Per iniziare a tracciare un progetto con Git:

```
mkdir sito-corso
cd sito-corso
git init
```

`git init` crea un repository Git vuoto nella cartella corrente.

In pratica viene creata una directory nascosta `.git` che contiene gli oggetti, i riferimenti ai branch e i metadati del repository.

A questo punto il progetto è "sotto versionamento", ma non esiste ancora nessun commit.

Esempio di avvio reale:

```
echo "# Sito corso FISM" >
README.md
git add README.md
git commit -m "Aggiunge
README iniziale"
```

Dopo il primo commit, il repository ha una storia iniziale e un branch attivo.

Leggere bene git status

Il comando che va usato continuamente

`git status` è il comando che ti dice in che stato si trova il progetto.
Ti permette di vedere:

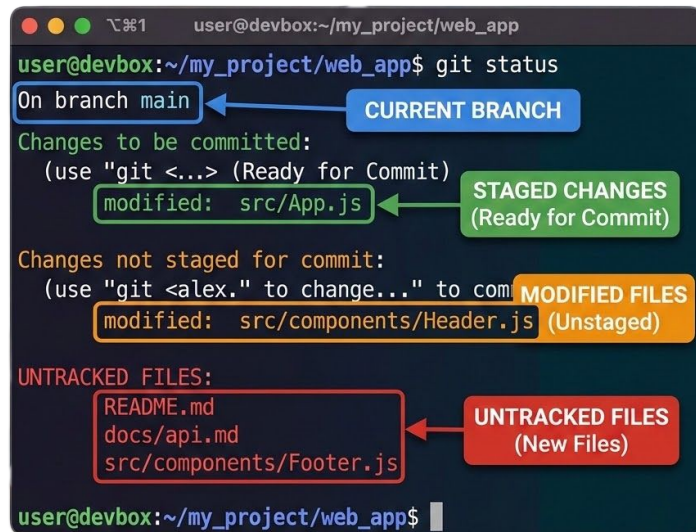
- file modificati ma non ancora in staging;
- file già in staging per il prossimo commit;
- file non tracciati;
- eventuali conflitti o situazioni anomale;
- il branch corrente.

Da usare sempre:

- prima di fare `add`;
- prima di fare `commit`;
- dopo un merge;
- quando "non capisci cosa sta succedendo".

Messaggio da martellare:
se sei perso, fai `git status`.

Git distingue chiaramente ciò che sarebbe incluso in un commit immediato da ciò che invece richiederebbe prima un `git add`.



```
user@devbox:~/my_project/web_app$ git status
On branch main
Changes to be committed:
  (use "git <...> (Ready for Commit)" to update what will be committed)
    modified:   src/App.js
Changes not staged for commit:
  (use "git <alex.> to change..." to update what will be committed)
    modified:   src/components/Header.js
UNTRACKED FILES:
  README.md
  docs/api.md
  src/components/Footer.js
```

The screenshot shows the output of the `git status` command in a terminal window. The output is color-coded and annotated with boxes and arrows:

- On branch main**: A blue box labeled "CURRENT BRANCH" points to this line.
- Changes to be committed:**: A green box labeled "STAGED CHANGES (Ready for Commit)" points to the "modified: src/App.js" line.
- Changes not staged for commit:**: An orange box labeled "MODIFIED FILES (Unstaged)" points to the "modified: src/components/Header.js" line.
- UNTRACKED FILES:**: A red box labeled "UNTRACKED FILES (New Files)" points to the list of files: "README.md", "docs/api.md", and "src/components/Footer.js".

Staging: selezionare il prossimo commit

Cosa fa davvero `git add`

`git add` non “salva definitivamente” il file: lo mette nello **staging**.

Punto importante:

- `git add` registra nello staging il contenuto del file **nel momento esatto** in cui esegui il comando. Se poi modifichi di nuovo il file, quelle nuove modifiche non entreranno nel commit a meno che tu non faccia di nuovo `git add`.
- Questo permette commit più puliti e ragionati. Non è obbligatorio fare un commit di tutto ciò che stai toccando: puoi scegliere cosa includere.

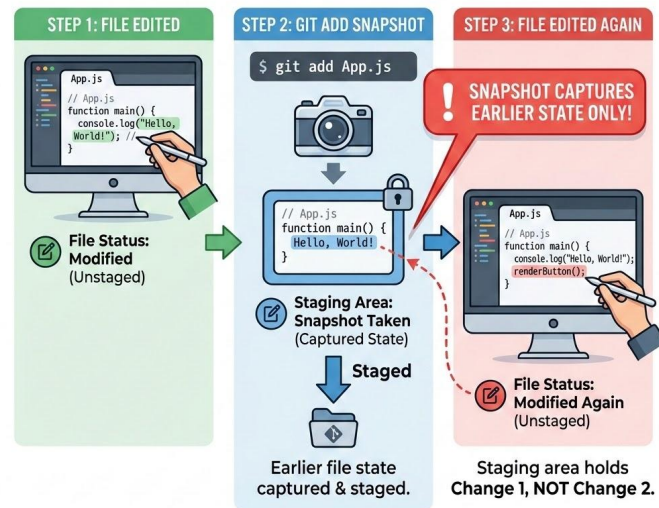
Esempio guidato:

Hai modificato `README.md` e `index.html`.

Vuoi fare prima il commit della documentazione e poi quello della pagina.

Puoi aggiungere e committare separatamente.

```
git add README.md
git add .
git add src/index.html
```



git commit: salvare uno stato coerente

Scrivere commit sensati

`git commit` crea un nuovo commit usando il contenuto attualmente in staging.

Un buon commit deve essere:

- piccolo o almeno coerente;
- leggibile nella cronologia;
- focalizzato su una singola modifica o gruppo logico di modifiche.

Regola pratica:

un commit dovrebbe poter essere capito da chi non era accanto a te mentre lo facevi.

Git definisce il commit come la creazione di un nuovo snapshot basato sui contenuti dell'index, cioè dello staging.

Messaggi deboli:

- fix
- update
- varie
- cose

Messaggi migliori:

- Corregge link errato alla documentazione
- Aggiunge pagina iniziale del progetto
- Aggiorna README con istruzioni di avvio



```
git commit -m "Aggiunge sezione installazione al README"
```

git log: leggere il passato del repository

Vedere la storia del progetto

Per vedere la **cronologia**: `git log`

Per una vista più leggibile:

`git log --oneline --graph --decorate`

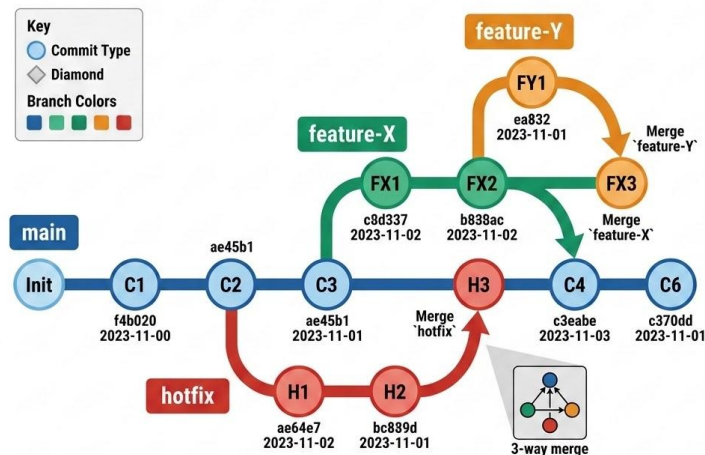
`git log` mostra i commit in ordine cronologico inverso per default: prima i più recenti.

Questo ti permette di:

- vedere l'evoluzione del progetto;
- capire quando è stata introdotta una modifica;
- seguire branch e merge;
- leggere messaggi di commit e hash.

Esempi di casi d'uso:

- "Quando abbiamo aggiunto il README?"
- "Qual è il commit prima del merge?"
- "Quanti commit sensati abbiamo fatto oggi?"



Da Git a GitHub

Git: sistema di controllo versione

GitHub: piattaforma di hosting per repository Git

Vantaggi di GitHub:

- Hosting remoto (backup e accessibilità)
- Strumenti di collaborazione
- Issue tracking
- Pull requests
- Integrazione con CI/CD
- Community e social coding



Alternative a GitHub

GitLab: open source, self-hosted o cloud

Bitbucket: integrazione con altri prodotti Atlassian

Azure DevOps: integrazione con l'ecosistema Microsoft

Gitea/Gogs: alternative leggere e self-hosted

Differenze principali: funzionalità, prezzi, privacy, integrazione



GitLab

— www.DavideCio.ME —

Repository locale vs repository remota

Locale e remoto non sono la stessa cosa

Una repository **locale** è quella sul tuo computer.

Una repository **remota** è una copia ospitata altrove, per esempio su GitHub.

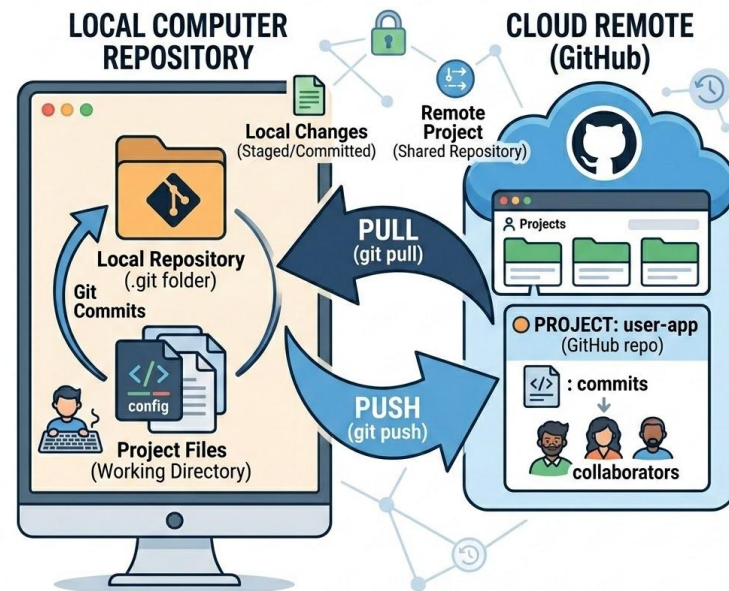
Perché serve una repo remota?

- backup del progetto;
- collaborazione tra più persone;
- revisione delle modifiche;
- accesso da più computer;
- pubblicazione pubblica o condivisione controllata.

Git può funzionare anche senza GitHub.

GitHub però aggiunge strumenti di collaborazione: issue, pull request, review, visibilità del progetto, pagine statiche, template, documentazione e altro.

Git è il motore. GitHub è una piattaforma di collaborazione.



clone, push, pull

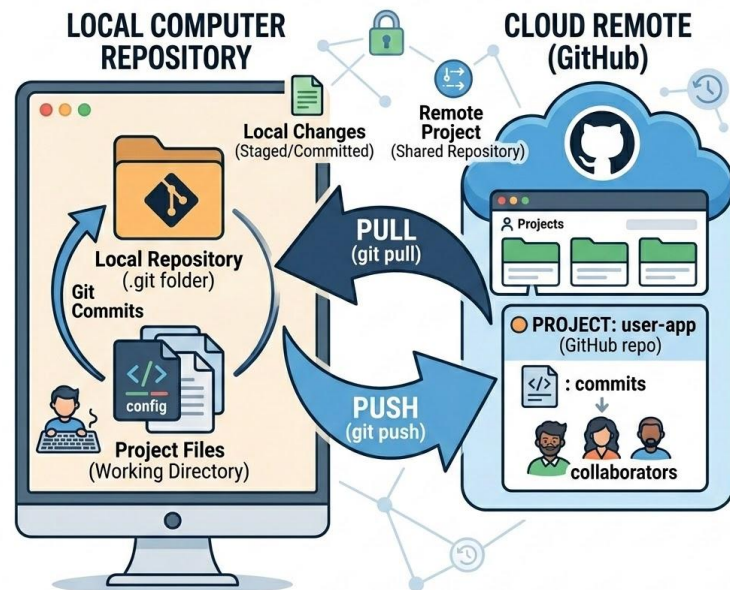
I tre verbi base del lavoro con il remoto

- **git clone**
Copia una repository remota in una nuova directory locale e imposta i riferimenti remoti.
 - `git clone https://github.com/utente/progetto.git`
- **git push**
Invia i commit locali verso il remoto.
 - `git push origin main` (il nome `origin` è il nome standard del remoto principale)
- **git pull**
Recupera e integra i cambiamenti dal remoto nel branch corrente.
 - `git pull origin main`

Spiegazione intuitiva:

- **clone** = porto il progetto da remoto a locale;
- **push** = mando i miei commit locali sul remoto;
- **pull** = porto in locale gli aggiornamenti del remoto.

Il clone crea una working tree locale e anche i remote-tracking branch relativi alla repository clonata. Il pull combina recupero e integrazione dei cambiamenti.



Creare una repository su GitHub

Dal progetto locale al progetto condiviso

Flusso semplice consigliato in aula:

1. creo la cartella locale;
2. faccio `git init`;
3. creo `README.md`;
4. faccio il primo commit;
5. creo una repository vuota su GitHub;
6. collego il remoto;
7. faccio il primo push.

Comandi tipici:

```
git remote add origin https://github.com/utente/nome-repo.git
git branch -M main
git push -u origin main
```

Alternativa:

- creare prima la repo su GitHub e poi fare `git clone`.
- usare GitHub Desktop
- creare la repo in locale e pubblicarla usando github CLI
- Usare un IDE con connessione a github (VS code, Antigravity, JetBrains, Zed, etc...)

(Su GitHub puoi anche aggiungere una licenza direttamente dall'interfaccia, scegliendo un template di file `LICENSE`.)



```
# Se non ce l'hai ancora, installa gh:
```

```
# macOS: brew install gh
```

```
# Linux: sudo apt install gh
```

```
# Login (una volta sola)
```

```
gh auth login
```

```
# Crea la repo e collegala automaticamente
```

```
gh repo create mio-progetto --public --source=. --
remote=origin --push
```

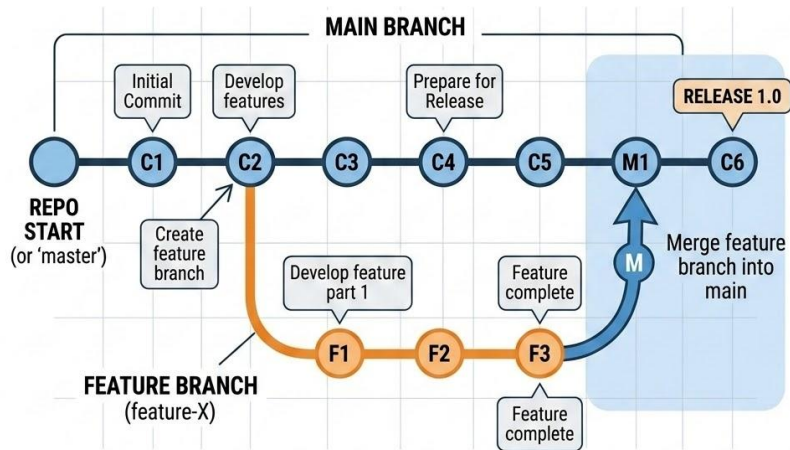
Branch: lavorare senza rompere tutto

Creare rami per modifiche isolate

Un branch è una linea di sviluppo separata. Serve per lavorare a una nuova funzionalità, correzione o esperimento senza sporcare subito il branch principale. Ed è utile per:

- riduce il rischio di features invalidanti;
- rende leggibile la storia;
- facilita la collaborazione;
- separa lavoro finito da lavoro in corso.

Git documenta i branch come riferimenti a diverse linee di sviluppo, e il merge integra la cronologia di un branch nell'altro



Comandi base:

`git branch feature-readme`

`git switch feature-readme`

oppure in una riga:

`git switch -c feature-readme`

Workflow tipico:

- `main` contiene la versione stabile;
- creo `feature-homepage`;
- lavoro lì;
- faccio commit;
- quando il lavoro è pronto, integro con merge o pull request.

Merge e conflitti

Riunire il lavoro dei branch

Quando una modifica è pronta, possiamo integrare il branch nel branch principale:

```
git switch main  
git merge feature-readme
```

Il merge prova a unire automaticamente le differenze.

Se due persone hanno modificato la stessa parte dello stesso file in modi incompatibili, Git segnala un **merge conflict**.

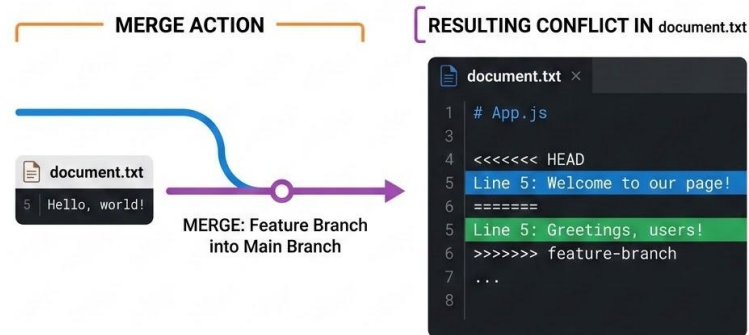
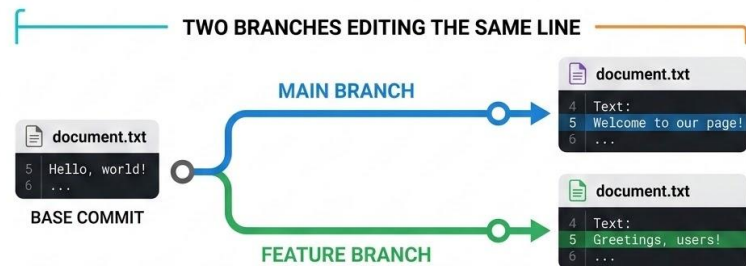
Cosa fare in caso di conflitto:

1. aprire il file;
2. leggere i marcatori di conflitto;
3. decidere quale contenuto mantenere o come combinarlo;
4. salvare il file corretto;
5. fare `git add`;
6. completare il commit di merge se richiesto.

Messaggio chiave:

il conflitto non è un disastro; è Git che ti chiede di decidere.

`git merge` incorpora in modo esplicito la storia di un altro branch nella cronologia corrente, e viene usato anche dal meccanismo di `git pull`.



Cosa fare quando sbagli

Errori frequenti e recupero minimo

Errori tipici dei principianti:

- hai modificato un file ma non volevi;
- hai messo in staging il file sbagliato;
- hai fatto un commit sbagliato;
- vuoi annullare l'effetto di un commit già fatto;
- non capisci più lo stato del repository.

Importante:

- `restore` lavora sul contenuto dei file;
- `reset` spesso viene usato per cambiare lo stato dello staging o del branch;
- `revert` non cancella la storia: aggiunge un nuovo commit che annulla gli effetti di uno precedente.

Git stesso distingue esplicitamente reset, restore e revert come tre operazioni diverse con scopi diversi.

```
#Ripristinare un file nella working directory
git restore nomefile

#Togliere un file dallo staging
git reset nomefile

#Creare un nuovo commit che annulla un commit precedente
git revert HASH
```

README, LICENSE e .gitignore

I file fondamentali in una repo seria

Una repository decente non contiene solo codice.

File minimi raccomandati:

- **README.md** → spiega cosa è il progetto e come usarlo;
- **LICENSE** → chiarisce i diritti d'uso;
- **.gitignore** → evita di versionare file inutili o sensibili.

Esempi di cose da ignorare con **.gitignore**:

- file temporanei;
- cache;
- cartelle di build;
- credenziali;
- file del sistema operativo.
- virtual env.

Molti caricano tutto alla cieca, poi si ritrovano repository sporche o addirittura con file che non andavano pubblicati.

GitHub raccomanda README e licenza per aiutare gli altri a capire il progetto e contribuire in modo corretto.

REPOSITORY FILE TREE (ROOT)

project/

README.md Documentation

Provide project intro, install steps, and usage guide.

assets/

images/

fonts/

LICENSE MIT License

State legal conditions for using this code.

docs/

api.md

src/

index.js

styles.css

.gitignore Git ignore rules

Specify file patterns for which Git should untrack.

package.json

yarn.lock

Struttura minima di un buon README

Come scrivere un README.md utile

Un README non deve essere lungo per forza. Deve essere utile.

Struttura minima consigliata:

1. Nome del progetto
2. Descrizione breve
3. Obiettivo del progetto
4. Tecnologie usate
5. Istruzioni di installazione o avvio
6. Esempio di utilizzo
7. Autori o gruppo
8. Licenza

GitHub descrive il README come il file che spiega perché il progetto è utile, cosa permette di fare e come usarlo. Inoltre l'interfaccia genera automaticamente una outline quando il markdown ha più heading.

```
# Mini sito corso FISM
```

```
Piccolo progetto didattico per imparare Git e GitHub.
```

```
## Obiettivo
```

```
Creare una repository collaborativa con README, branch e GitHub Pages.
```

```
## Avvio
```

```
Aprire il file index.html nel browser.
```

```
## Autori
```

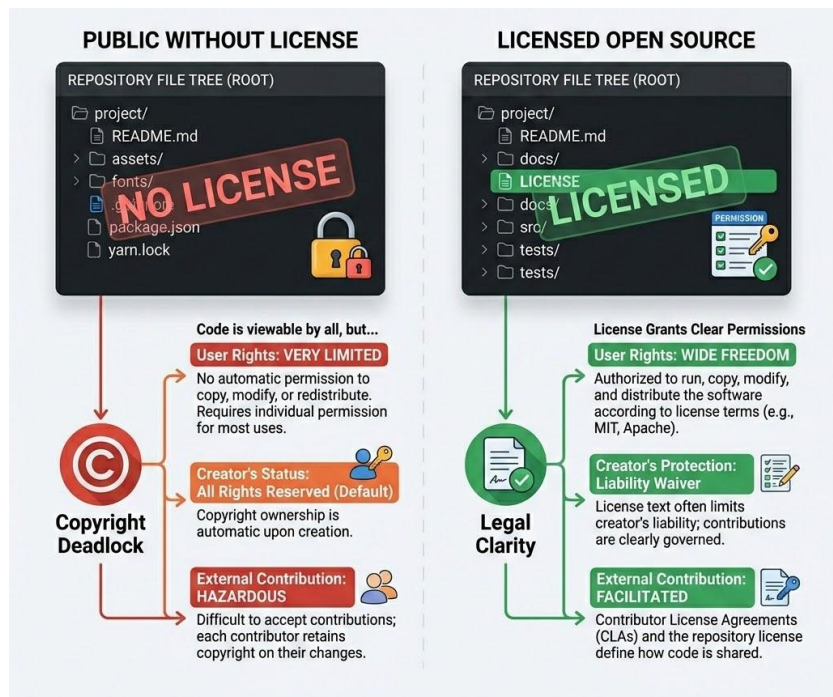
```
Gruppo 2
```

```
## Licenza
```

```
MIT
```


Licenze: la panoramica pratica che serve davvero

Se pubblichi codice, devi decidere con quali permessi



Pubblicare codice online **non** significa automaticamente renderlo libero da usare come si vuole.

Se una repository non ha una licenza, per default si applicano i normali diritti d'autore: l'autore mantiene tutti i diritti e gli altri non possono riprodurre, distribuire o creare opere derivate senza permesso.

GitHub chiarisce anche che mettere un repository pubblico consente agli altri di vederlo e fare le azioni permesse dal servizio, come il forking, ma questo non sostituisce una vera licenza open source.

Traduzione pratica:

- repo pubblica senza licenza $\neq$ software open source;
- se vuoi che altri possano usare, modificare e ridistribuire, devi dichiararlo.

Quattro casi da saper distinguere

Proprietaria, MIT, GPL, Apache 2.0

1. Proprietaria / nessuna licenza aperta

È il caso più restrittivo. L'autore mantiene i diritti e non concede libertà tipiche dell'open source.

2. MIT

Licenza permissiva: consente uso, copia, modifica, distribuzione, sublicenza e vendita, purché si mantengano avviso di copyright e licenza.

3. GPL

Copyleft forte. Puoi usare e modificare liberamente, ma se distribuischi il tuo software derivato, devi rilasciarlo con la stessa licenza GPL (effetto "virale"). La v3 aggiunge protezioni contro le patent war. Esiste anche la LGPL (Lesser GPL): come la GPL ma permette il collegamento con software proprietario, usata per librerie.

4. Apache 2.0

Anche questa è permissiva, ma in più include una concessione esplicita di brevetto da parte dei contributori, cosa utile soprattutto in contesti più strutturati (i contributori ti cedono anche i diritti sui loro patent relativi al codice).

Scelta pratica in aula:

- MIT se vuoi massima semplicità;
- GPL se vuoi imporre che le derivate restino aperte;
- Apache 2.0 se vuoi permissiva + attenzione esplicita ai brevetti.
- per casi professionali o commerciali seri serve valutazione più attenta 🍀🍀🍀.

	PROPRIETARY	MIT	GPL	APACHE 2.0
CORE PRINCIPLE	 CLOSED SOURCE	 PERMISSIVE	 COPYLEFT	 PERMISSIVE + PATENTS
KEY PERMISSIONS	 ✓ VIEWABLE CODE (optional) ✓ NOT modifiable ✓ NOT redistributable ✓ NOT for derivatives.	 ✓ USE for commercial/non-commercial ✓ MODIFY ✓ REDISTRIBUTE (Merge without attribution for derivatives).	 ✓ VIEW code ✓ MODIFY code ✓ REDISTRIBUTE (MUST open-source derivatives) ✓ Same license (Viral) propagation.	 ✓ USE ✓ MODIFY ✓ REDISTRIBUTE ✓ PATENT LICENSE GRANT (crucial) ✓ Explicit attribution (notices) required.
CREATOR PROTECTION	 ALL RIGHTS RESERVED (Default) • Full Control • High Liability.	 • NO LIABILITY • NO WARRANTIES.	 NO LIABILITY (for creators of base work) WARRANTY (none implied).	 LIABILITY LIMITATION (standard clause) • Explicit Waiver.
EXAMPLE USE CASES	 • Windows OS • Adobe Photoshop • Many commercial apps.	 • jQuery • Babel • Many small libraries.	 • Linux Kernel • Git • GIMP • WordPress.	 • Kubernetes • Docker • TensorFlow • React Native.

Issue: discutere e tracciare lavoro

A cosa servono le issue su GitHub

Le issue servono per pianificare, discutere e tracciare lavoro.

Non sono solo **"bug report"**: possono rappresentare idee, richieste di nuove funzionalità, attività da fare, dubbi, problemi di documentazione o decisioni da prendere.

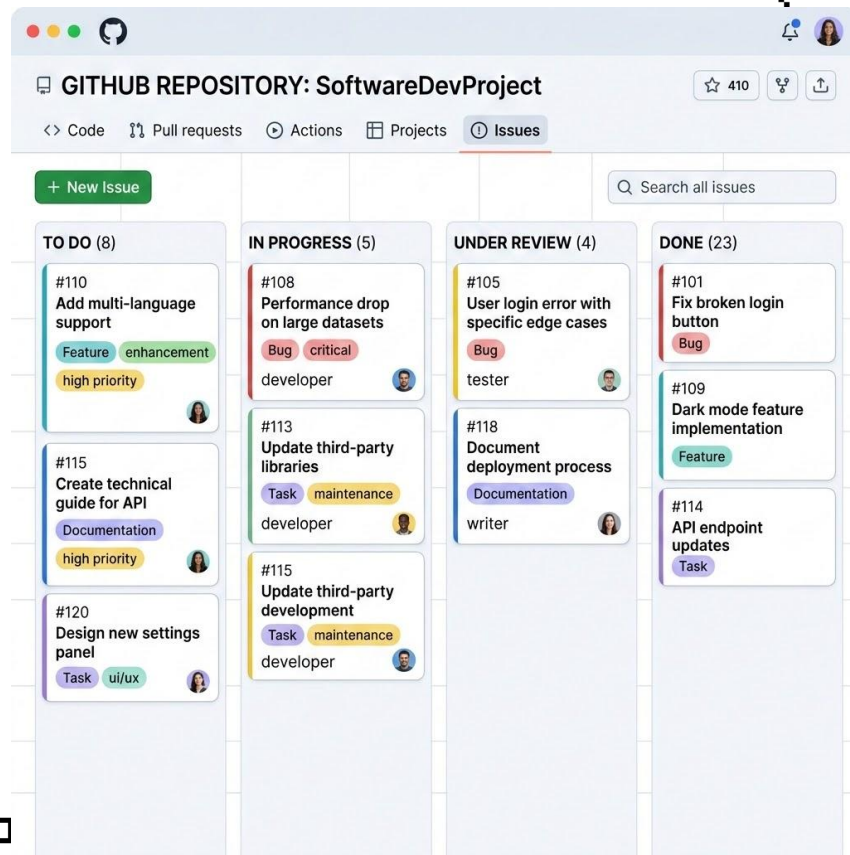
Esempi:

- "Aggiungere sezione installazione nel README"
- "Correggere link non funzionante nella home"
- "Creare pagina contatti"
- "Scegliere la licenza della repository"

Vantaggi:

- il lavoro non resta disperso in chat;
- c'è un riferimento condiviso;
- si può assegnare, commentare, chiudere;
- il progetto diventa più leggibile anche per chi arriva dopo.

GitHub definisce le issue come strumenti rapidi e flessibili per pianificare, discutere e tracciare lavoro.



www.Datocia

Come funziona una PR

Pull request: proporre modifiche in modo serio

Una pull request è una proposta di integrazione di modifiche in un progetto. Essenzialmente è una proposta di merge...

Le modifiche vengono presentate da un branch e possono essere discusse, riviste e approvate prima del merge.

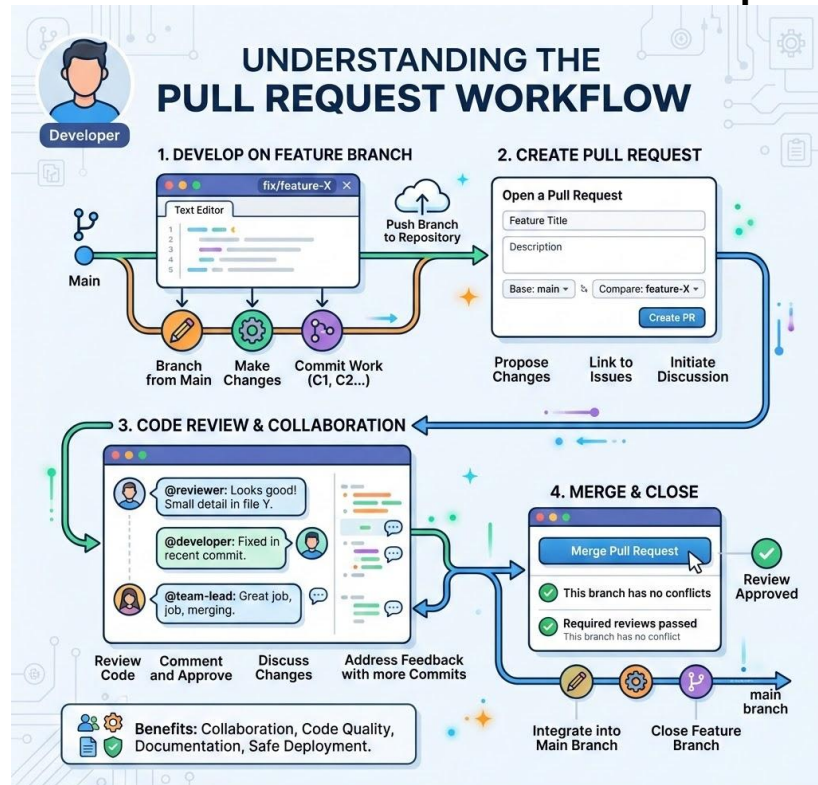
Perché è utile:

- evita merge alla cieca;
- consente revisione del lavoro;
- rende visibili commenti e motivazioni;
- migliora la qualità del progetto.

Workflow minimo:

1. creo branch;
2. faccio commit;
3. push del branch su GitHub;
4. apro pull request;
5. review o discussione;
6. merge.

GitHub presenta le pull request come la funzionalità centrale della collaborazione: permettono di proporre cambiamenti, discuterli e fare review prima dell'integrazione. Chiunque abbia almeno accesso in lettura a una repo può aprire una PR; se non ha permessi di scrittura, può passare da un fork.



Workflow collaborativo semplice

Collaborazione a coppie e contributi a progetti online

Opzione A – un solo repo condiviso

- uno crea la repo;
- invita il compagno come collaborator;
- entrambi clonano;
- ognuno lavora su un branch;
- si apre una PR interna.

Opzione B – modello open source

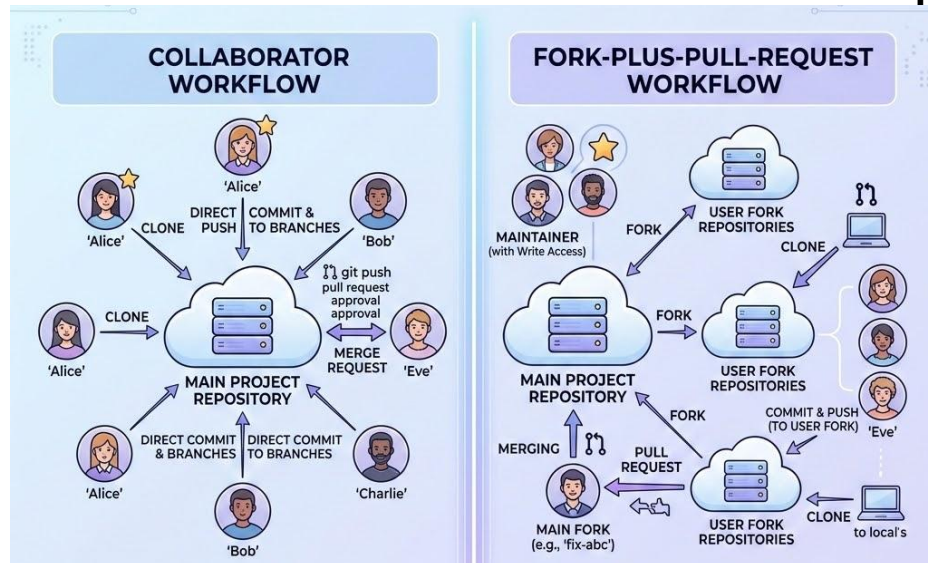
- uno crea il progetto principale;
- l'altro fa fork;
- lavora sul fork;
- apre pull request verso il repository originale.

Quando contribuire a progetti online:

1. leggi README e licenza;
2. controlla issue aperte;
3. segui eventuali linee guida di contributo;
4. lavora su branch o fork;
5. apri una PR chiara e piccola.

GitHub definisce il fork come una copia personale collegata all'upstream, utile per sperimentare e proporre modifiche senza toccare direttamente il repository originale.

- opzione B modello realistico del mondo open source



GitHub Pages

Pubblicare una semplice pagina web dal repository

GitHub Pages permette di pubblicare un sito statico direttamente da una repository GitHub.

Due modi principali:

- pubblicazione da un branch;
- pubblicazione tramite GitHub Actions.

Per la didattica di oggi conviene la strada semplice:

deploy da branch, usando la root del branch oppure una cartella `/docs`.

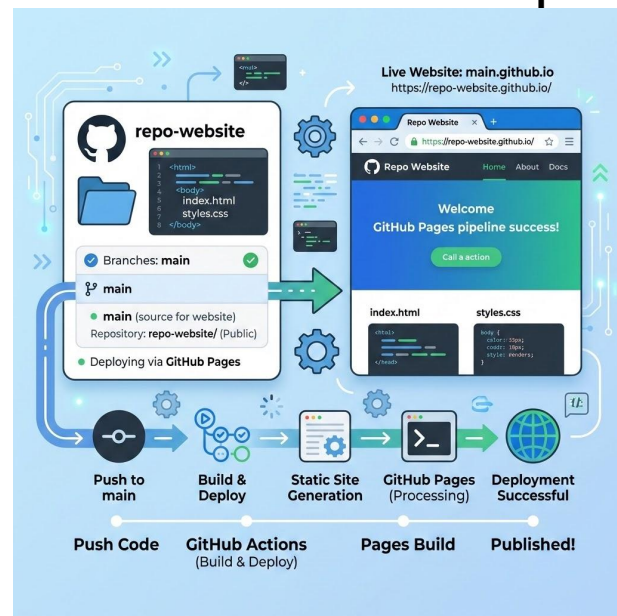
Caso base didattico:

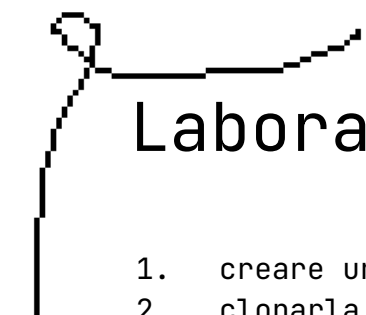
- repository con `index.html`;
- Settings → Pages;
- Source → Deploy from a branch;
- scegli `main` e `/root`, oppure `main` e `/docs`;
- salvi;
- GitHub pubblica il sito.

Attenzione utile:

- se il sito è generato con qualcosa di diverso da Jekyll, GitHub raccomanda spesso un workflow Actions oppure di disabilitare la build Jekyll con un file `.nojekyll` nel source pubblicato.

Per questa lezione basta pubblicare una pagina HTML minimale con titolo, descrizione del progetto e link al repository.



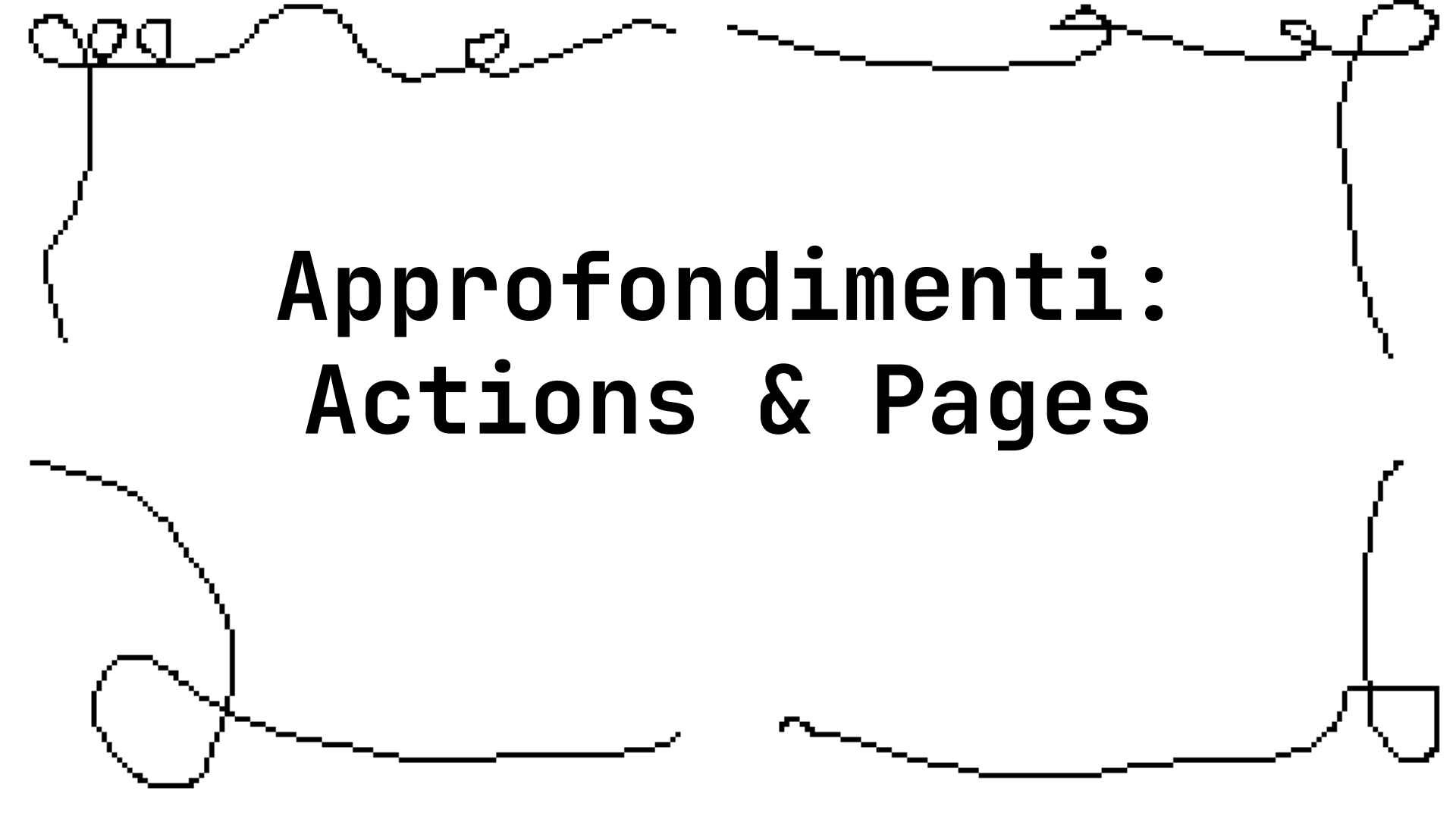


Laboratorio guidato

1. creare una repository GitHub;
2. clonarla oppure collegare il repo locale al remoto;
3. creare `README.md`;
4. fare il primo commit;
5. creare un branch;
6. modificare README o una pagina HTML;
7. fare almeno altri 2-3 commit
8. integrare il branch con merge o pull request;
9. facoltativo: pubblicare una pagina con GitHub Pages.

Checklist minima finale:

- repo attiva su GitHub;
- README iniziale presente;
- almeno 3-4 commit leggibili;
- eventuale pagina pubblicata.

A hand-drawn, pixelated black border surrounds the text. It features various loops, swirls, and a small square at the bottom right corner.

Approfondimenti: Actions & Pages



GitHub Actions

Componenti principali:

- Workflows (file YAML in `.github/workflows/`)
- Jobs
- Steps
- Actions

Trigger: push, pull request, schedule, manuale, ecc.

Esempio base di GitHub Actions

```
1 name: CI
2
3 on:
4   push:
5     branches: [ main ]
6   pull_request:
7     branches: [ main ]
8
9 jobs:
10  build:
11    runs-on: ubuntu-latest
12
13    steps:
14      - uses: actions/checkout@v3
15
16      - name: Setup Node.js
17        uses: actions/setup-node@v3
18        with:
19          node-version: '16'
20
21      - name: Install dependencies
22        run: npm ci
23
24      - name: Run tests
25        run: npm test
```

Casi d'uso comuni per GitHub Actions

Build e test automatici

- Compilazione del codice
- Esecuzione di test unitari e di integrazione
- Analisi statica del codice

Deployment automatico

- Siti web
- Applicazioni mobili
- Container Docker

Automazione di repository

- Gestione di issue e PR
- Generazione di documentazione
- Rilascio di versioni



GitHub Pages

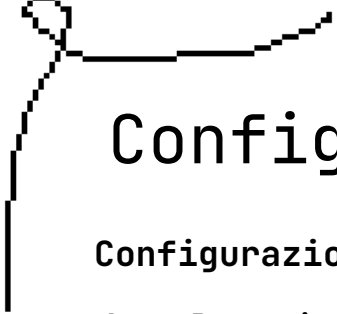
Cos'è: hosting gratuito per siti statici direttamente da un repository GitHub

Tipi di GitHub Pages:

- Pages di utente/organizzazione (username.github.io)
- Pages di progetto (username.github.io/project)

Caratteristiche:

- HTTPS incluso
- Supporto per domini personalizzati
- Ottimizzazione della cache



Configurare GitHub Pages



Configurazione base:

1. Repository pubblico su GitHub
2. Impostazioni → Pages
3. Seleziona branch e cartella (root o /docs)

Soluzioni comuni:

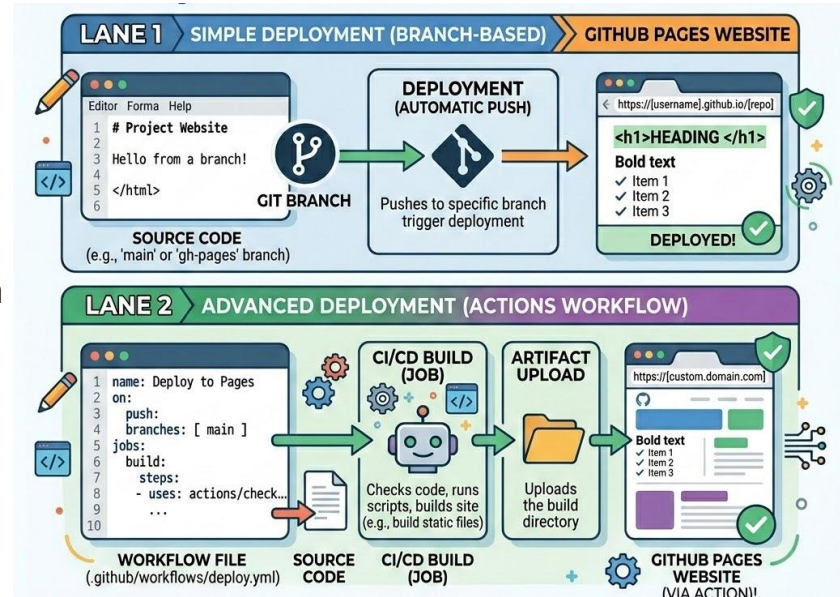
- HTML/CSS/JS statici
- Altri generatori di siti statici (Hugo, Next.js, ecc.) con GitHub Actions

Pages con Actions

Per siti statici molto semplici basta pubblicare da branch.

Ma quando il sito deve essere costruito con un generatore statico o una pipeline custom, GitHub Pages può usare GitHub Actions per build e deploy.

La documentazione GitHub indica chiaramente che, se non serve controllo sul processo di build, il deploy da branch è il modo più semplice; se invece il sito richiede una build personalizzata, il workflow Actions diventa l'approccio corretto



A hand-drawn decorative border in black ink, featuring various loops, swirls, and flourishes that frame the central text. The border is irregular and artistic, with some elements resembling small circles or teardrops.

Approfondimento: Markdown

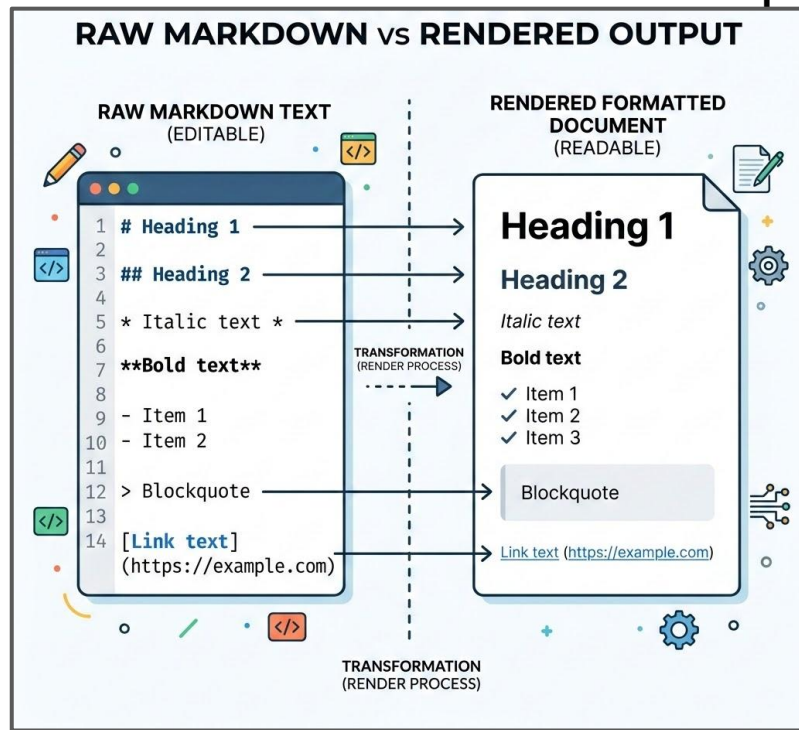
Che cos'è Markdown

Markdown è un linguaggio di markup leggero basato su testo semplice.

Serve a scrivere contenuti formattati senza usare editor complessi: invece di cliccare pulsanti grafici, si usano simboli molto semplici da tastiera per indicare titoli, liste, link, codice, citazioni e altre strutture del testo.

- è leggibile anche come testo puro (un `.docx` aperto come testo è inutilizzabile; un `.md` invece si capisce quasi subito anche grezzo)
- si scrive velocemente;
- funziona bene dentro Git;
- rende facile confrontare le modifiche;
- è perfetto per README, documentazione, issue, pull request, wiki e appunti tecnici.

Su GitHub, Markdown è alla base di moltissimi contenuti testuali: README, commenti, issue, pull request e wiki. GitHub organizza questa scrittura attraverso la propria sintassi di base e varie estensioni di formattazione.

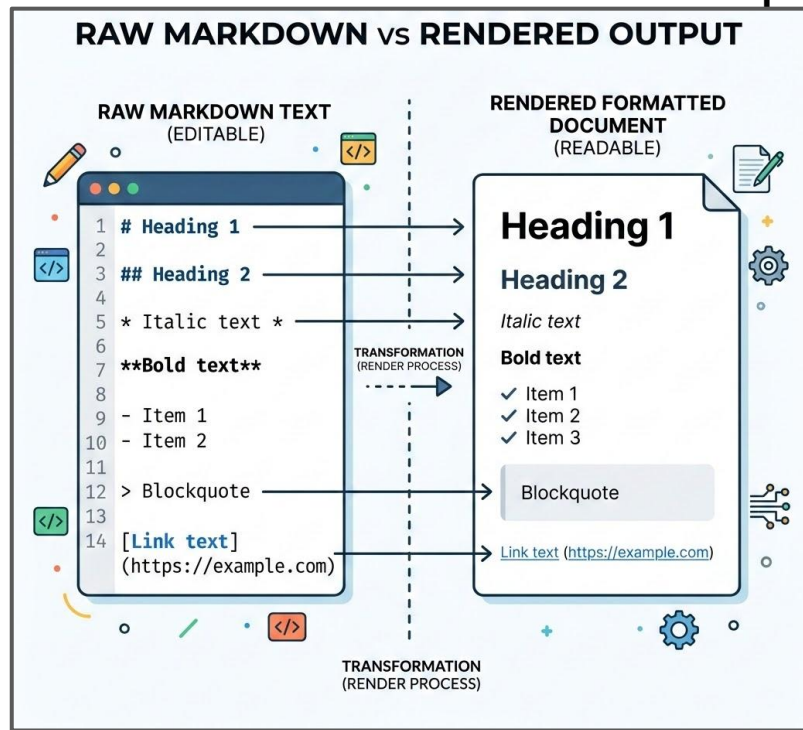


Sintassi minima indispensabile

Le strutture minime più utili da conoscere in Markdown sono:

- **Titoli**
 - # Titolo principale
 - ## Sezione
 - ### Sottosezione
- **Grassetto e corsivo**
 - ****grassetto****
 - **corsivo**
- **Liste puntate e numerate**
 - - voce uno
 - - voce due
 - 1. primo
 - 2. secondo
- **Link**
 - [GitHub](https://github.com)
- **Codice inline**
 - Usa ``git status`` per controllare lo stato del repository.

Per una lezione introduttiva, queste cinque strutture bastano già per scrivere un README dignitoso. GitHub documenta esplicitamente l'uso di backtick singoli per il codice inline e dei blocchi separati per il codice vero e proprio.



Altre tre cose utilissime nei README

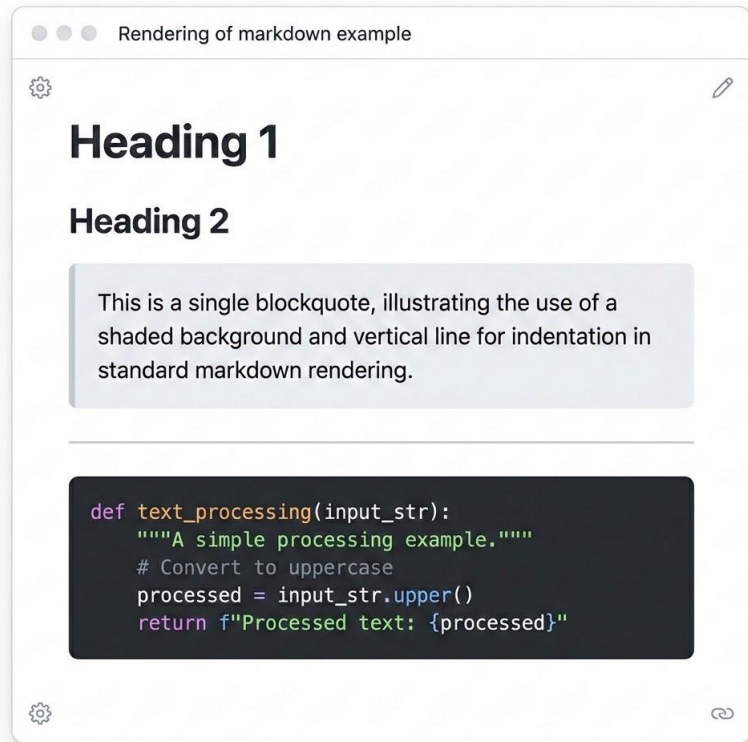
Oltre alle strutture base, ci sono tre strumenti molto utili:

- **Blocco di codice**
 - `bash`
 - `git status`
 - `git add .`
 - `git commit -m "Primo commit"`
- **Citazione**
 - `> Questo progetto è stato creato durante il corso FISM.`
- **Separatore orizzontale**
 - `---`

I blocchi di codice sono essenziali quando vuoi mostrare:

- comandi da terminale;
- pezzi di configurazione;
- frammenti HTML, CSS o JavaScript;
- output di esempio.

GitHub supporta i fenced code blocks con tripli backtick, e la documentazione raccomanda anche di lasciare una riga vuota prima e dopo per rendere il sorgente più leggibile. Inoltre i code block possono avere syntax highlighting indicando il linguaggio.



Le estensioni utili su GitHub

Su GitHub si usa normalmente **GitHub Flavored Markdown (GFM)**, cioè una variante con estensioni utili in contesto collaborativo.

Tra le caratteristiche più pratiche:

- **tabelle** per organizzare informazioni;
- **task list** con checkbox;
- **autolink** automatici a URL, issue, pull request e commit;
- **sezioni collassabili** con `<details>`;
- **blocchi di codice** con **evidenziazione sintattica**.

Esempi:

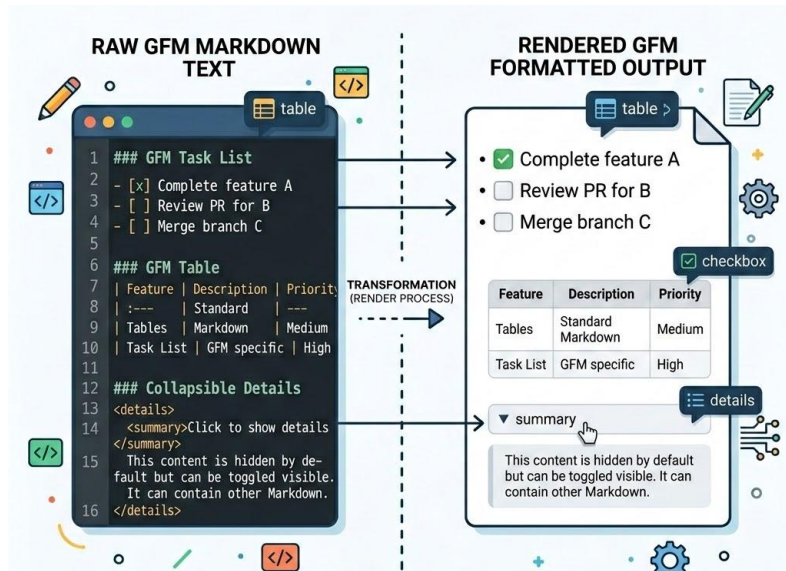
- **Tabella**

```
| File | Descrizione |
|-----|-----|
| README.md | Presentazione del progetto |
| index.html | Pagina iniziale |
```

- **Task list**

- [x] Creare repository
- [x] Scrivere README
- [] Pubblicare con GitHub Pages

GitHub documenta esplicitamente tabelle, task list, sezioni collassabili, syntax highlighting e autolink come parte delle sue funzionalità di scrittura avanzata. La specifica GFM definisce anche formalmente i task list item con la sintassi `[ ]` e `[x]`.



Esempio completo di mini README in Markdown

```
# Mini progetto Git e GitHub

Piccolo progetto didattico realizzato durante il corso
FISM 2026.

## Obiettivo
Imparare a usare Git, GitHub, branch, pull request e
documentazione minima.

## File principali
- `README.md`: descrizione del progetto
- `index.html`: pagina iniziale del sito
- `LICENSE`: licenza del progetto

## Avvio
Apri `index.html` nel browser.

## Attività completate
- [x] Primo commit
- [x] Creazione branch
- [ ] GitHub Pages

## Autori
Gruppo 1
```

Errori tipici in Markdown

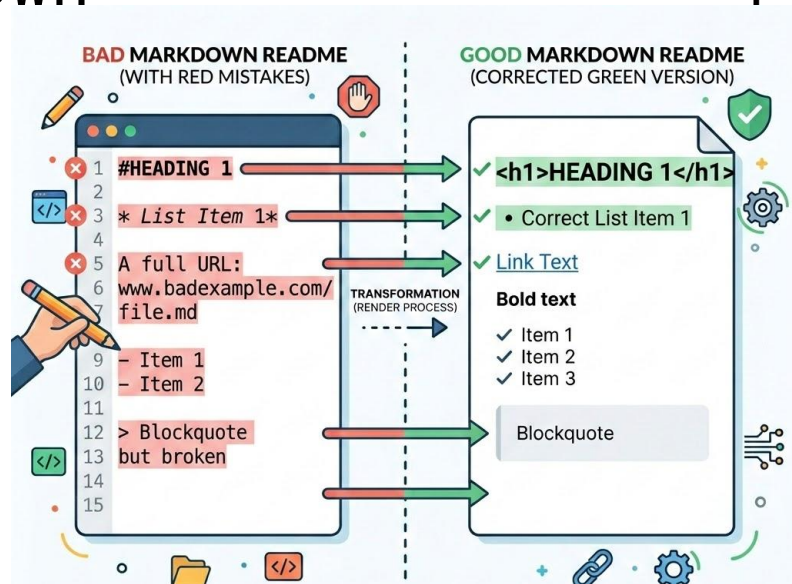
Errori frequenti dei principianti:

- dimenticare lo spazio dopo `#` nei titoli;
- rompere una tabella con colonne non allineate;
- usare troppi livelli di titolo inutili;
- mettere testo tecnico senza blocchi di codice;
- scrivere README lunghissimi ma poco leggibili;
- mischiare testo normale e comandi terminale senza distinguerli;
- abusare del grassetto.

Regola pratica:

un README deve essere **chiaro prima che bello**.

Su GitHub la resa finale dipende dalla sintassi usata correttamente; in particolare code block, tabelle e task list sono molto più leggibili quando la struttura del sorgente è pulita.



Markdown avanzato: <details>, autolink e riferimenti

Quando la documentazione cresce, non conviene lasciare tutto aperto e piatto.

Strumenti utili:

Sezione collassabile

```
<details>
  <summary>Mostra istruzioni avanzate</summary>
```

Qui puoi inserire dettagli extra, log, spiegazioni lunghe o note tecniche.

```
</details>
```

Autolink su GitHub

- URL normali diventano link automaticamente;
- riferimenti a issue, pull request e commit possono diventare link interni molto comodi.

Queste funzionalità aiutano a scrivere documentazione più ordinata, soprattutto in README più lunghi, issue tecniche e pull request articolate. GitHub documenta sia l'uso di `<details>` per le sezioni collassabili sia gli autolink automatici per URL, issue, pull request e commit.

Collapsible Details:

This section is an example of a collapsible block, using `<details>` and `<summary>` tags.

```
<br/>
```

```
</details>
```

Task Lists & Issue References:

You can create task lists and reference issues:

- [x] Implement new feature ([#123](#))
- [] Fix bug ([#124](#) - In Progress)
- [] Write documentation ([#125](#))

```
<br/>
```

Notice how issue numbers like [#123](#), [#124](#), [#125](#) are automatically linked to their respective issues within the repository.

Code Blocks with Syntax Highlighting:

Markdown Example

```
This is some italic and bold text.
You can also use `inline code`.
```

Markdown avanzato: Mermaid e diagrammi

GitHub supporta anche diagrammi scritti in forma testuale tramite blocchi di codice dedicati.

Il caso più interessante da mostrare è **Mermaid**.

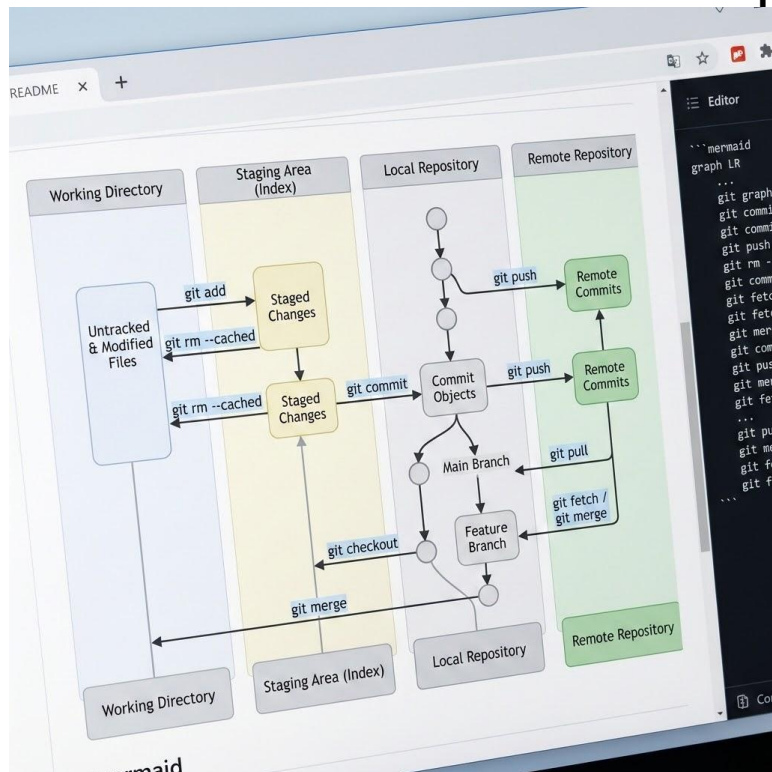
Esempio:

```
```mermaid
graph TD;
 A[Working Directory] --> B[Staging Area];
 B --> C[Repository];
```
```

Perché è interessante:

- il diagramma resta versionabile come testo;
- si può modificare facilmente;
- le differenze si vedono bene in Git;
- è utile per documentare workflow, architetture e processi.

GitHub documenta esplicitamente Mermaid come strumento supportato nei fenced code blocks, capace di renderizzare flow chart, sequence diagram, pie chart e altri tipi di diagramma.



A hand-drawn decorative border in black ink, featuring various loops, swirls, and flourishes that frame the central text. The border is irregular and artistic, with some elements resembling calligraphic flourishes.

ESERCIZI

Errori tipici in Markdown

Esercizio 1 – Primo repository locale

Creare una cartella di progetto, inizializzarla con Git, aggiungere un README e fare due commit distinti:

- uno per il titolo del progetto;
- uno per una sezione "Obiettivo".

Cosa controllare

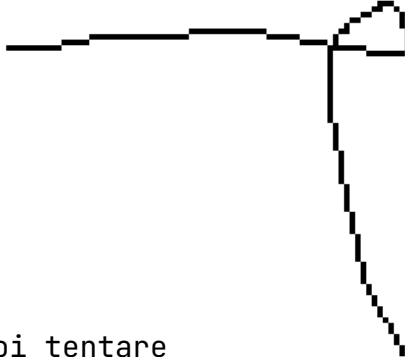
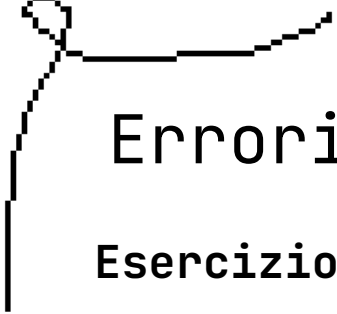
- uso corretto di `git init`;
- uso corretto di `git add`;
- messaggi di commit leggibili.

Esercizio 2 – Branch e merge

Creare un branch chiamato `feature-presentazione`, modificare il README con una nuova sezione, tornare su `main` e fare merge.

Cosa controllare

- branch creato correttamente;
- cronologia leggibile;
- merge eseguito nel branch giusto.



Errori tipici in Markdown

Esercizio 3 – Piccolo conflitto

A coppie, far modificare la stessa riga del README in due branch diversi e poi tentare il merge.

Esercizio 4 – GitHub remoto

Pubblicare la repo su GitHub, eseguire `push`, far aggiungere una modifica dal compagno, poi fare `pull`.

Errori tipici in Markdown

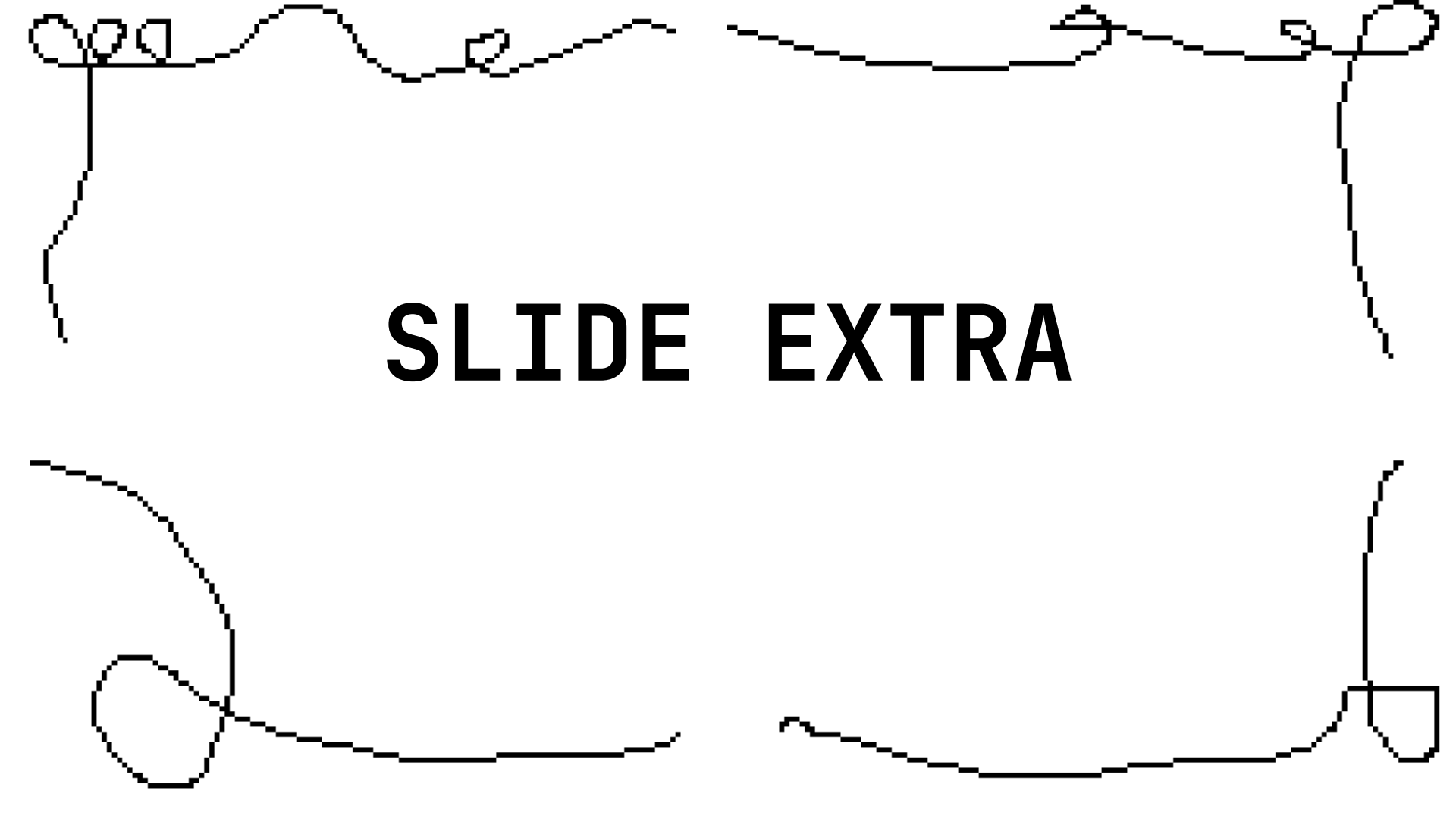
Esercizio 5 — Collaboration flow

Aprire una issue, creare un branch collegato a quella attività, fare modifica, aprire PR, commentarla e chiuderla con merge.

Esercizio 6 — GitHub Pages

Creare `index.html` molto semplice e pubblicarlo con GitHub Pages.

```
<!doctype html>
<html lang="it">
<head>
  <meta charset="utf-8">
  <title>Mini progetto GitHub Pages</title>
</head>
<body>
  <h1>Mini progetto corso FISM</h1>
  <p>Repository realizzata durante la lezione
su Git e GitHub.</p>
</body>
</html>
```

A hand-drawn decorative border in black ink, featuring various loops, swirls, and flourishes that frame the central text. The border is composed of several distinct sections: a top-left corner with a small loop, a top-right corner with a larger loop, a bottom-left corner with a large oval, and a bottom-right corner with a small square-like loop. The lines are slightly irregular, giving it a sketchy, artistic feel.

SLIDE EXTRA

git fetch vs git pull

Recuperare cambiamenti: differenza tra fetch e pull

Molti principianti imparano subito `git pull`, ma è utile sapere che non è l'unica operazione possibile.

git fetch

- scarica aggiornamenti dal remoto;
- aggiorna i riferimenti remoti locali;
- **non** integra automaticamente i cambiamenti nel branch corrente.

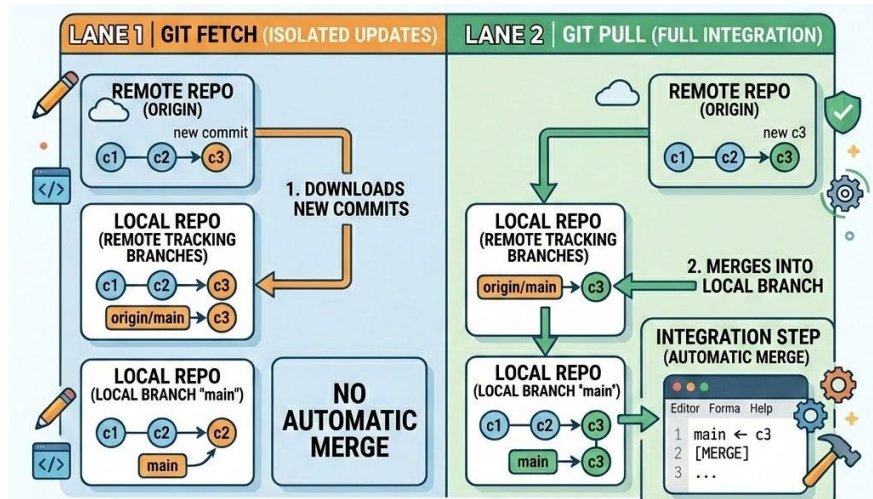
git pull

- recupera i cambiamenti dal remoto;
- poi li integra nel branch corrente.

In pratica:

- `fetch` = "fammi vedere cosa è cambiato";
- `pull` = "porta qui e prova anche a integrare".

Questo è utile quando vuoi controllare con calma prima di mischiare il tuo lavoro con quello altrui. GitHub stessa nel glossario distingue `fetch` da `pull`, indicando che `fetch` aggiunge i cambiamenti dal remoto senza impegnarli automaticamente nel branch locale, mentre `pull` recupera e integra



Merge vs Rebase

Due modi diversi di integrare la storia

Quando due linee di sviluppo devono essere riunite, i due concetti da conoscere sono:

Merge

- mantiene visibile la biforcazione della storia;
- crea, quando serve, un commit di merge;
- è più esplicito e spesso più semplice da capire.

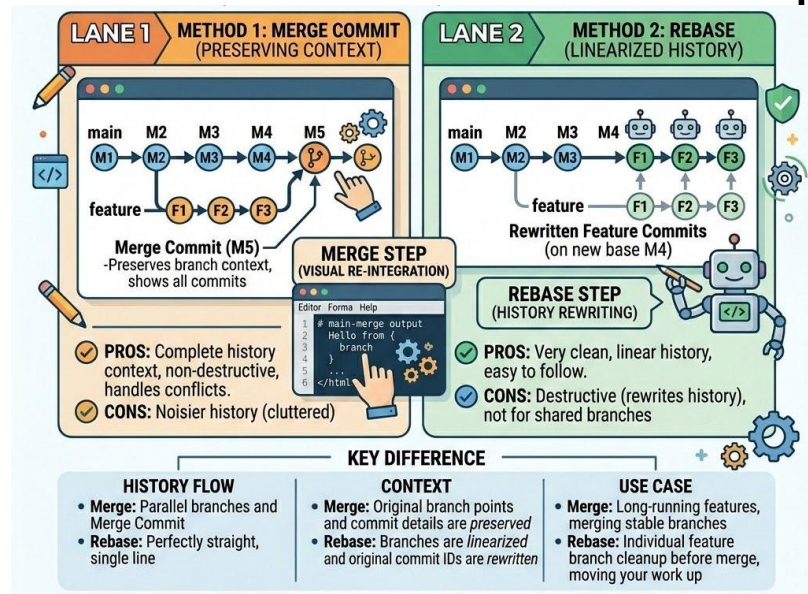
Rebase

- riscrive la posizione dei commit di un branch come se fossero partiti da un altro punto;
- produce una cronologia più lineare;
- richiede più attenzione, perché altera la storia dei commit.

Per una classe base conviene usare soprattutto **merge**.

rebase è un approfondimento utile da nominare, ma non da rendere obbligatorio nel primo vero laboratorio.

GitHub Docs tratta **merge** e **rebase** come strumenti distinti della gestione della storia Git; il rebase è presentato come una riscrittura della base dei commit, quindi va usato con più consapevolezza.



A hand-drawn decorative border in black ink, featuring various loops, swirls, and flourishes that frame the central text. The border is irregular and artistic, with some elements resembling calligraphic flourishes.

ESERCIZI EXTRA

Esercizi - Intermedi

1. Lavorare con i branch

- a. Crea un nuovo branch chiamato "feature-login", aggiungi un file login.txt con del contenuto, committa le modifiche, torna al branch main e poi unisci (merge) il branch feature-login in main.

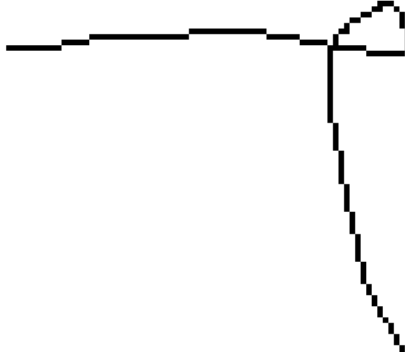
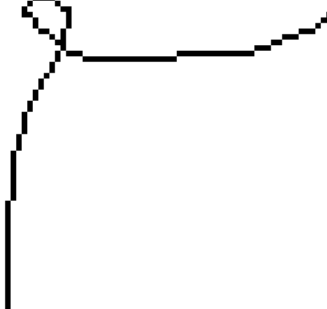
2. Risolvere un conflitto di merge

- a. Crea un branch "feature-profile", modifica il README.md aggiungendo una sezione "Profilo utente", torna al branch main, modifica la stessa parte del README.md in modo diverso, prova a fare il merge e risolvi il conflitto.

Risorse per continuare ad imparare

Risorse per continuare ad imparare

- **Documentazione ufficiale:**
 - git-scm.com/doc
 - docs.github.com
- **Esercitazioni interattive:**
 - learngitbranching.js.org
 - lab.github.com
- **Libri consigliati:**
 - "Pro Git" di Scott Chacon (gratuito online)
 - "GitHub Actions: Up & Running" di Rosemary Wang
- **Community e forum:**
 - GitHub Community Forum
 - Stack Overflow



FINE



— WWW.DRACO.ME —